



System design

Modal Dialog

★ Premium



Medium



30 mins



866 done

Report an issue 🛠️

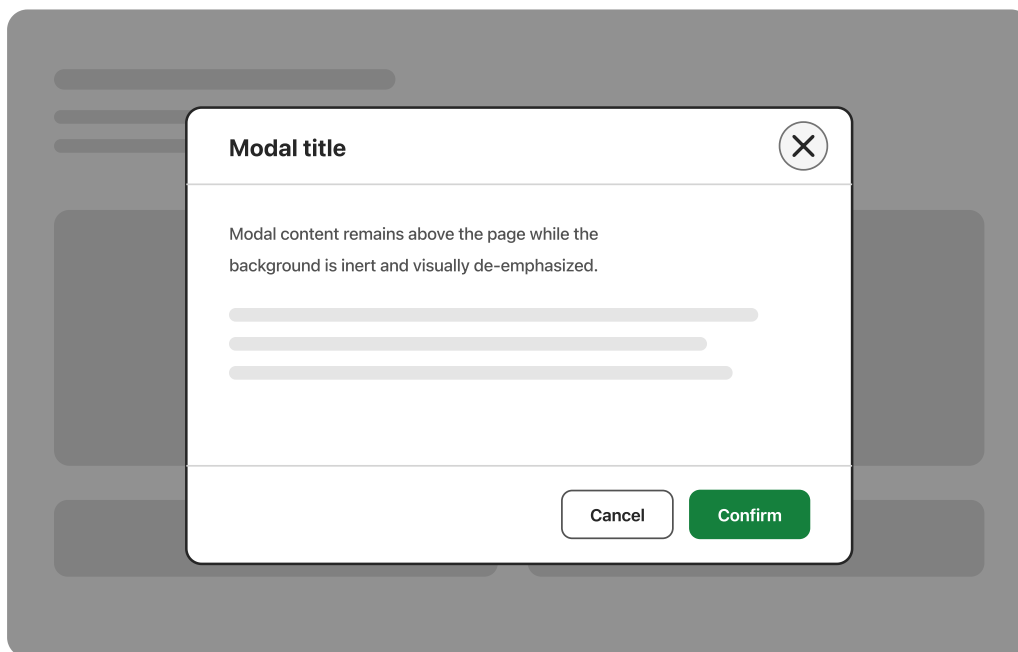
Asked at these companies



Tiktok

Question

Design a modal/dialog component that shows content in a window overlaying the page.



Real-life examples

- Modal · Bootstrap v5.3

- React Modal component - Material UI
- Dialog — Base UI

Requirements exploration

Before designing the API, we scope the component by clarifying which parts it owns, how much the user can customize it, and which devices and content patterns it must support.

What are the components of the modal?

It's up to you to decide. At the basic level, it should allow customization of the contents. Whether to add built-in support for close buttons, titles, and footer actions will be up to you.

What amount of flexibility does the user have in customizing the design?

Users should be able to customize colors, fonts, padding, etc., of the modal elements to match their branding.

What devices will this component be used on?

All devices: mobile, tablet, desktop.

Exercise Beta

Which requirements define the reusable modal component before its architecture is chosen? Select all that apply.

1 Decide whether consumers provide arbitrary content or use built-in title, body, close, and footer regions. ☐

2 Require every modal to use one fixed title, body, and footer template with no consumer-provided content. ☐

3

Support modal content on mobile, tablet, and desktop viewports.

☐

4

Allow consumers to adapt colors, fonts, spacing, and other presentation to their product.

☐



Reveal answer

Check answer ↩

Architecture / high-level design

Modals, like many components that reveal content, have a trigger element and content elements. Modals can be opened through user actions or background actions, so we should decouple the trigger source from the modal contents.

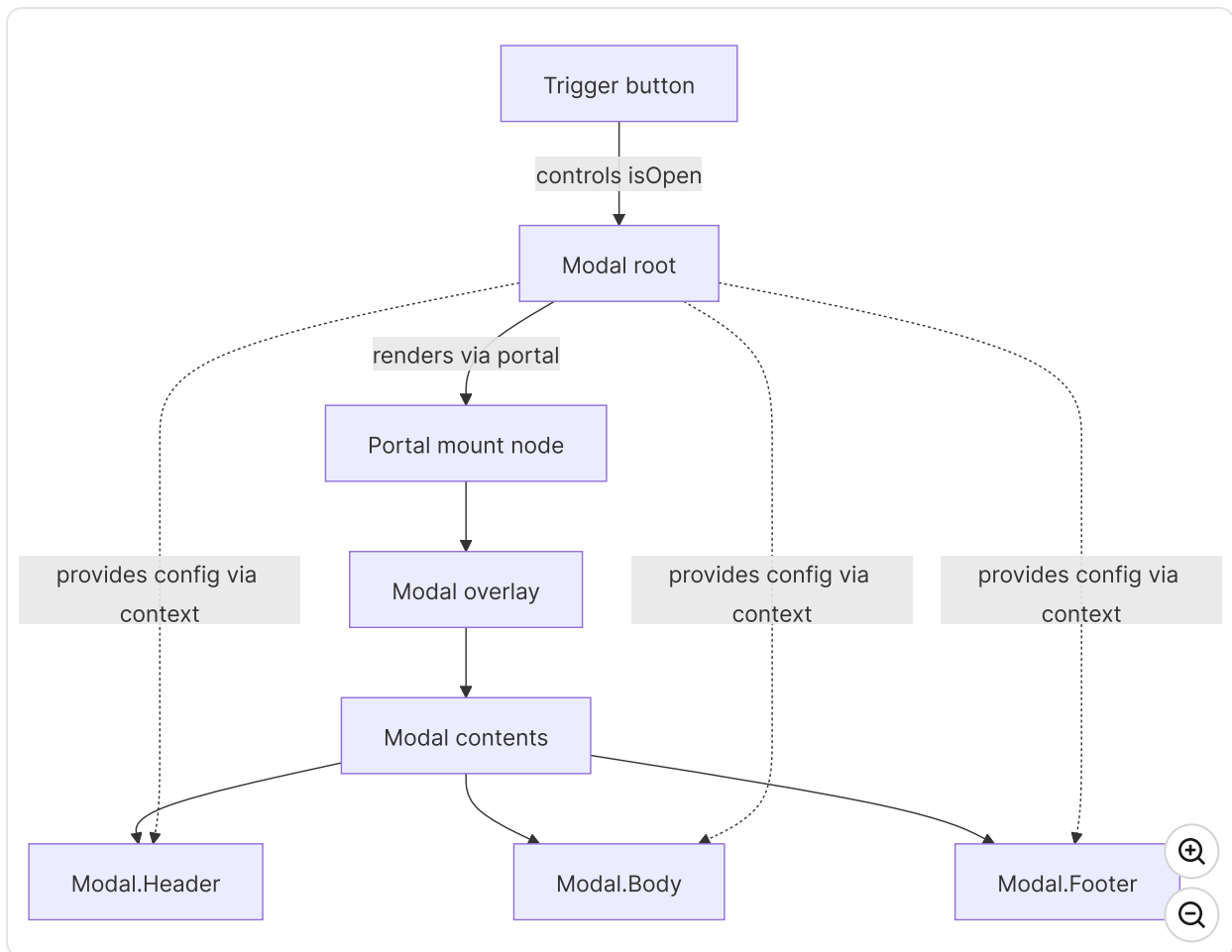
Basic modals that only include the contents don't have a complicated architecture. However, many modals from UI libraries have three distinct sections: header, body, and footer.

An example usage of modal components in React, with event handlers omitted:

```
1  <Modal>
2    <Modal.Header>
3      <Modal.Title>My Modal Title</Modal.Title>
4      <Modal.Close aria-label="Close" />
5    </Modal.Header>
6    <Modal.Body>
7      <p>Modal body text goes here.</p>
8    </Modal.Body>
9    <Modal.Footer>
10     <button>Close</button>
11     <button>Confirm</button>
12   </Modal.Footer>
13 </Modal>
```

Component	Role
Modal Root (<code>Modal</code>)	Root component, coordinates events between the inner components.
Modal Overlay	Component that renders the background overlay, usually dimming out the page contents.
Modal Header (<code>Modal.Header</code>)	The top section of the modal; contains the title and a close button.
Modal Body (<code>Modal.Body</code>)	The main contents of the modal.
Modal Footer (<code>Modal.Footer</code>)	Optional bottom section of the modal; usually contains close/submit buttons.

In React, components can communicate with their parents using context or props. We choose to use context here since we are using a composition model and passing props is inconvenient. `<Modal>` should contain a context provider that provides the config options (`<Modal>` 's `props`) to all its child components in case they need them.

*Modal component hierarchy and context flow***Exercise** Beta

Which architecture keeps modal triggers decoupled from content while coordinating the component's regions?

- 1 Keep content open state explicitly but infer whether the overlay and controls are active from their current CSS visibility.
- 2 Let a modal root coordinate open state and configuration for composed overlay, header, body, footer, and close parts.
- 3 Let the overlay, header, body, footer, and close control each maintain their own open state.
- 4 Require every trigger to own and render a private copy of the complete modal content tree.

[Reveal answer](#)[Check answer ↩](#)

Data model

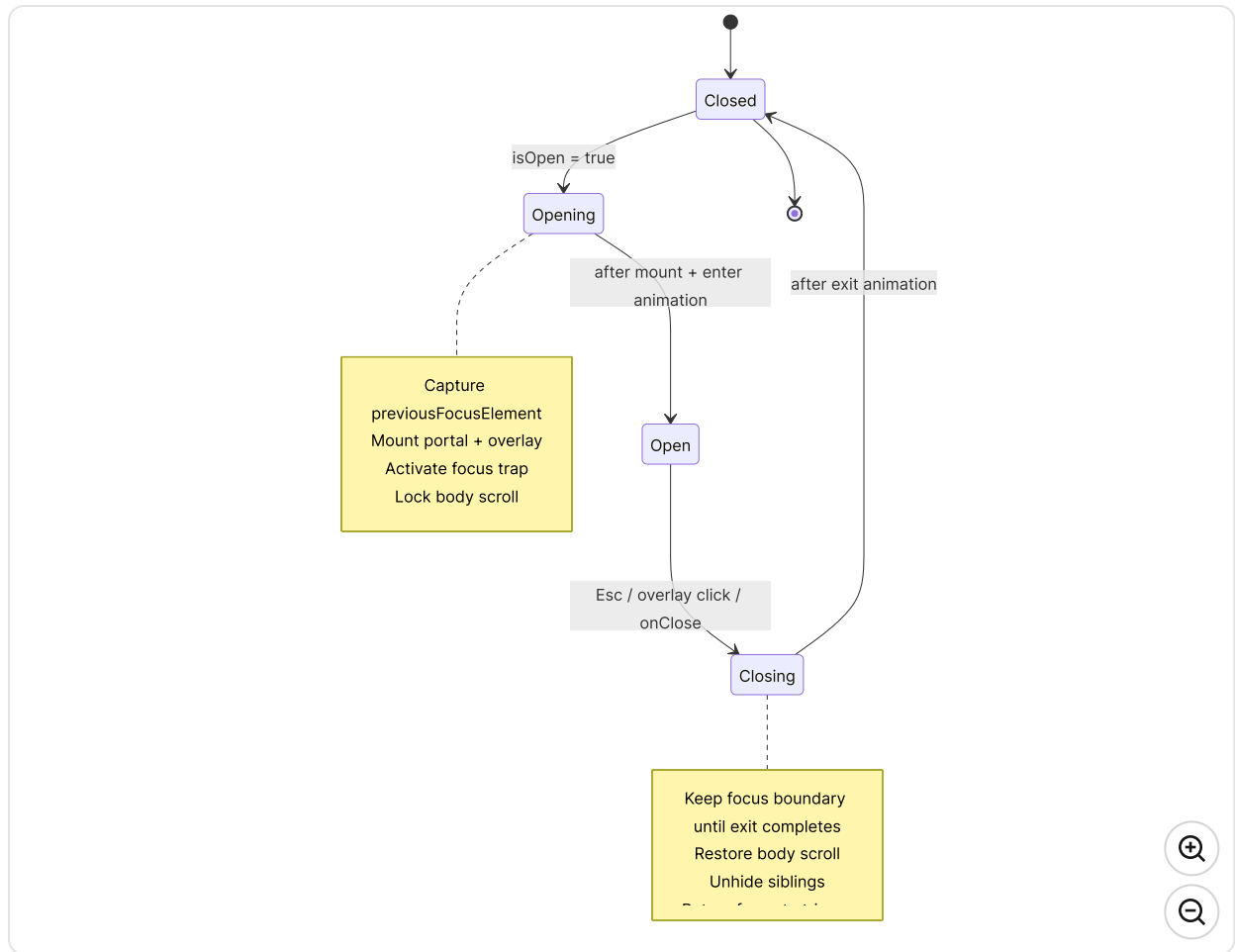
Note that for designing components, it might make sense to design the interface/API first, or both the data model and API concurrently. It depends on the component at hand. Feel free to jump between the two sections.

Modal components do not need much state. We'll build the modal as a controlled component, which is the usual approach taken by libraries. Hence, the open/closed state is managed outside the component.

State	Type	Description
<code>previousFocusElement</code>	<code>HTMLElement</code>	The DOM element in focus before the modal was shown. Read more on why this is needed in the Focus Management section.
<code>phase</code>	<code>'closed' 'opening' 'open' 'closing'</code>	Keeps the dialog mounted and inertness/focus effects active through an exit transition.
<code>layerId</code>	<code>string</code>	Stable manager entry used to determine which stacked modal owns dismissal and focus.

See below for configuration options which are also part of the data model.

Even though the persisted state is tiny, the modal moves through several observable phases during its lifecycle. Tracking them explicitly clarifies when side effects like the portal mount, focus trap, scroll lock, and background inertness should be set up and torn down.



Modal lifecycle from closed to open and back

Exercise Beta

A modal can animate closed and can open another dialog above it. Which values belong in its coordinated lifecycle state? Select all that apply.

1 Stack position and topmost status when multiple dialogs are supported. ☐

2 Use the body's overflow value as the shared signal for whether any modal is open. ☐

3 Store the previous active element's serialized markup and recreate it after close if the original was removed. ☐

4 The element or strategy used to restore focus after closing. ☐

5 Open state and, when animations require it, a distinct closing phase. ☐

[Reveal answer](#)[Check answer ↩](#)

Interface definition (API)

The API is split into props shared across subcomponents and props specific to each composition piece (root, overlay, header, body, footer).

General props

These props apply to most of the components.

Prop	Type	Description
<code>children</code>	<code>React.Node</code>	Children of the component. If using TypeScript/Flow, you can enforce specific components to be used as <code>children</code> .

Prop	Type	Description
<code>as</code>	<code>string Component</code>	In case there is a need to customize the underlying DOM element/component.
<code>className</code>	<code>string</code>	Classnames to add to components in case further visual customization is needed. May or may not be needed depending on theming approach.

Modal

Prop	Type	Description
<code>isOpen</code>	<code>boolean</code>	Whether the modal is open or closed.
<code>onClose</code>	<code>Function</code>	Callback that requests the parent to close the modal, such as after pressing the close button or hitting the "Escape" key.
<code>maxHeight</code>	<code>number undefined</code>	Max height of the modal. There should be a sensible default of around 80% of the viewport height.
<code>width</code>	<code>number undefined</code>	Width of the modal. There should be a sensible default of 500-600px.

Modal.Header

Basic version doesn't need props other than `children`.

`Modal.Title` and optional `Modal.Description` provide visible text whose stable IDs are wired to `aria-labelledby` and `aria-describedby`. `Modal.Close` routes activation through the same close transition as Escape and overlay dismissal.

Modal.Body

Basic version doesn't need props other than `children`.

Modal.Footer

Basic version doesn't need props other than `children`.

Customizing appearance

Guidance on designing good APIs for customizing UI components can be found in the [Front End Interview Guidebook's UI Components API Design Principles Section](#).

Most of the modal's contents in the header/body/footer will be provided by the user; hence, there isn't that much default styling a modal component needs to provide.

Exercise Beta

Which API balances modal flexibility with predictable behavior?

- 1 Accept one rendered content callback and infer title, description, and close controls from the resulting DOM.
- 2 Expose an overlay root element and require callers to append content and register their own Escape and focus handlers.
- 3 Expose a root with open/change callbacks plus header, body, footer, title, and close primitives.



Reveal answer

Check answer ↩

Optimizations and deep dive

The tricky parts of a modal are rendering outside the DOM hierarchy, focus management, accessibility, animations, localization, and stacked modal behavior:

- **Rendering**: This section explains portals, stacking contexts, overlays, scroll locking, and the rendering constraints of floating UI.
- **Accessibility (a11y)**: This section covers focus trapping, keyboard behavior, screen-reader semantics, and ARIA modal guidance.
- **Animations and transitions**: This section discusses entry and exit animations, independent overlay/content transitions, and reduced-motion concerns.
- **Internationalization (i18n)**: This section covers translated strings, text overflow, and direction-aware modal layouts.
- **Stacked modals**: This section explains how multiple modal layers affect focus, dismissal, z-index, and background inertness.
- **Advanced topics**: This section collects deeper modal concerns that may come up after the core design is covered.

The embedded sandboxes are simplified visual demonstrations of clipping, placement, dismissal, and animation. The surrounding prose defines the required naming, focus, topmost-click, and reduced-motion behavior.

Rendering

Modals float above the page, which creates two rendering challenges: escaping the parent's clipping and stacking contexts, and animating the modal's entry and exit smoothly.

Breaking out of DOM hierarchy

Rendering the modal is trickier than it seems due to the fact that modals are displayed over the page and do not follow the normal flow of page elements. It's important to render the modal outside of the DOM hierarchy of the parents, because if the parents contain styling that clips their contents, the modal contents might not be fully visible. Here's an example from the [React docs](#) demonstrating the issue.

In React, rendering outside the DOM hierarchy of the parent component can be achieved using React Portals. Common use cases of portals include tooltips, dropdown menus, and popovers.

★ **Use a portal for the interview implementation**

Portal the custom dialog into `document.body` or a documented overlay root so it escapes ancestor clipping and stacking contexts. The native top layer is a production alternative, discussed later, rather than the baseline implementation for this interview question.

Overlay

To help users focus on the content within the modal, there is usually an overlay/backdrop to dim out the page's contents. To render an element that covers the whole page, we can use the following CSS:

```
1 .modal-overlay {  
2   /* Black color with some opacity. */  
3   background-color: rgba(0, 0, 0, 0.7);
```

```
4    position: fixed;
5    top: 0;
6    left: 0;
7    right: 0;
8    bottom: 0;
9  }
```

Centered modal

To center the modal contents within the modal overlay, we can add the following styles to the `.modal-overlay`:

```
1  .modal-overlay {
2    /* Original styles are omitted. */
3    display: flex;
4    justify-content: center;
5    padding: 20px;
6  }
```

which will work with the following HTML structure.

```
<div class="modal-overlay">
  <div class="modal-contents">...</div>
</div>
```

If vertical centering of the contents is desired, `align-items: center` can be added to `.modal-overlay`.

Here's a basic example of a modal with an overlay and optional vertical center mode:

Maximum height

Use a shrinkable inline size such as `min(600px, calc(100vw - 2rem))` and a viewport-aware block-size cap. Keep the header and footer visible while `Modal.Body` owns vertical scrolling.

Scroll lock

When the modal is shown, the modal contents are in focus. To prevent the user from scrolling the background contents, the page should lock page-level scrolling. One way is to add `overflow: hidden` to the `<body>`.

✓ Lock body scroll whenever the modal is open

Without a scroll lock, scroll events bubble to the underlying page and the background quietly shifts behind the dialog, especially disorienting on touch devices where overlay taps and scroll gestures overlap. Toggle `overflow: hidden` on `<body>` (or equivalent) on open and restore the previous value on close so nested or reopened modals do not leak their lock.

Rendering in HTML or JavaScript

The modal can either be:

1. Rendered into the HTML like **Bootstrap's Modals**. The modal is initially hidden with `display: none` or the `hidden` attribute. `opacity: 0` alone does not remove it from focus, pointer interaction, or the accessibility tree.
2. Rendered on the fly via JavaScript after the modal trigger button is activated.

The pro of rendering in the HTML first is better runtime performance due to fewer DOM operations needed to show the modal. The downside is that the HTML can be unnecessarily bloated, especially if the modal is never shown at all. Since modal contents usually contain secondary information, they shouldn't contribute towards SEO and don't need to be server-side rendered. The benefits of rendering the modal beforehand in HTML are relatively minor.

Exercise 1 of 2 Beta

A modal opens from inside a clipped scrolling panel. Which rendering decisions keep the dialog visible and the background stable? Select all that apply.

- ☐ 1 Lock background scrolling while preserving the page's previous scroll position.
- ☐ 2 Use a fixed overlay that covers the viewport independently of the trigger panel's scroll position.
- ☐ 3 Render the dialog and backdrop through a portal into the document body or a documented overlay root.
- ☐ 4 Unmount the page beneath the first modal so background controls cannot receive focus.
- ☐ 5 Keep each modal near its trigger and choose a z-index larger than its closest positioned ancestor.



Reveal answer

Check answer ←

Accessibility (a11y)

Modals must be usable with mouse, keyboard, and assistive tech. The subsections below cover each input modality and the ARIA semantics that tie them together.

Mouse interactions

Typically, a completed click on the active modal's overlay requests close. Handle this on the backdrop itself and only for the topmost modal; pointer-down dismissal can fire before a user cancels a drag or scroll gesture.

```
1 function onOverlayClick(event) {
2   if (event.target === event.currentTarget && modalManager.isTopmost(id)
3     onClose();
4 }
5 }
```

Taken together with the keyboard and programmatic paths, the open and close flow touches the trigger, the portal, the focus trap, and the scroll lock in a specific order. The sequence below captures how these pieces fit together across a typical open-then-dismiss interaction.

```
Mermaid error: Parse error on line 15:
...; keep focus trapped P-->>R: Exit anim
-----^
Expecting '()', 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW',
'SOLID_ARROW_TOP', 'SOLID_ARROW_BOTTOM', 'STICK_ARROW_TOP',
'STICK_ARROW_BOTTOM', 'SOLID_ARROW_TOP_DOTTED', 'SOLID_ARROW_BOTTOM_DOTTED',
'STICK_ARROW_TOP_DOTTED', 'STICK_ARROW_BOTTOM_DOTTED',
'SOLID_ARROW_TOP_REVERSE', 'SOLID_ARROW_BOTTOM_REVERSE',
'STICK_ARROW_TOP_REVERSE', 'STICK_ARROW_BOTTOM_REVERSE',
'SOLID_ARROW_TOP_REVERSE_DOTTED', 'SOLID_ARROW_BOTTOM_REVERSE_DOTTED',
'STICK_ARROW_TOP_REVERSE_DOTTED', 'STICK_ARROW_BOTTOM_REVERSE_DOTTED',
'BIDIRECTIONAL_SOLID_ARROW', 'DOTTED_ARROW', 'BIDIRECTIONAL_DOTTED_ARROW',
'SOLID_CROSS', 'DOTTED_CROSS', 'SOLID_POINT', 'DOTTED_POINT', got 'NEWLINE'
```

Open and close flow across trigger, portal, and focus trap

Here's an example in React:

Focus management

The most complicated aspect of implementing a modal is probably focus management. Contents within modals are meant to be treated like a separate document; using the `Tab` key cycles through focusable elements within the dialog only, and focus can never be on elements outside of the component for as long as the modal is shown. This behavior/phenomenon is known as "focus trapping".

When a modal opens, focus moves to an element inside the modal. Generally, focus is set on the first focusable element, but there are some exceptions as mentioned in Dialog (Modal) pattern.

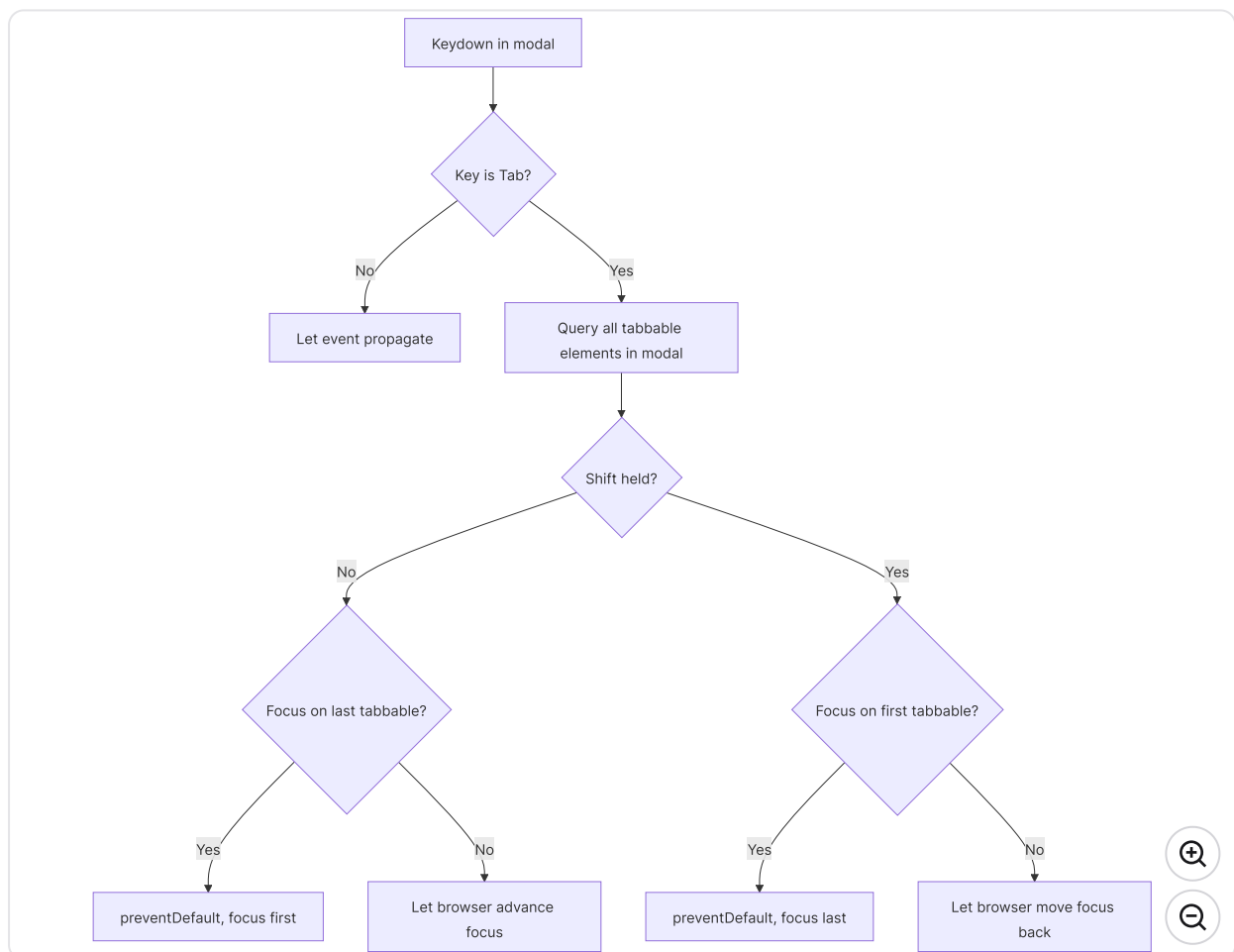
When a modal closes, focus returns to the element that opened the modal (unless the element no longer exists, in which case focus moves to another reasonable element).

How to implement focus management for modals:

1. When the modal is first opened, keep a reference to the element that opened the modal in the modal's state.

2. Focus on an element inside the modal.
3. Add `keydown` listeners for the `Tab` key that contain the following logic:
 1. When the `Tab` key is pressed, determine if the focus should be shifted to the next or previous tabbable element by checking if the `Shift` key is also pressed (via the `shiftKey` value on the `KeyboardEvent`).
 2. Find all tabbable elements within the modal.
 3. From the currently focused element, find the next/previous tabbable element.
 4. Focus on that new element.
4. When the modal is closed, hide the modal and move focus to the element that opened the modal.

The tricky part is the wraparound: the `Tab` handler has to detect the edges of the tabbable set and cycle focus back to the opposite end so it never escapes into background content. The flowchart below captures that decision for both `Tab` and `Shift + Tab` .



Focus trap wraparound for Tab and Shift+Tab

In practice, focus trapping can be done with libraries such as `focus-trap` or `react-focus-lock`.

! **Always trap focus inside the modal and return it to the trigger on close**

Skipping the focus trap lets Tab escape into the dimmed background, where keyboard and screen-reader users end up interacting with controls they cannot see. Failing to restore focus to the opening element on close is just as bad: it strands assistive tech at <body> and forces sighted keyboard users to tab back from the top of the page every time a dialog dismisses.

Keyboard interactions

The following is extracted from the Dialog (Modal) pattern:

Key	Description
Tab	Moves focus to the next tabbable element inside the modal. If focus is on the last tabbable element inside the modal, moves focus to the first tabbable element inside the modal.
Shift + Tab	Moves focus to the previous tabbable element inside the modal. If focus is on the first tabbable element inside the modal, moves focus to the last tabbable element inside the modal.
Esc	Closes the modal.

Focus trapping is essential for the required `Tab`-ing behavior; otherwise, the focus will "leak" out of the modal:

WAI-ARIA roles, states, and properties

The following is extracted from the Dialog (Modal) pattern.

- The element that serves as the modal container has a role of `dialog`.
- All elements required to operate the modal are descendants of the element that has role `dialog`.
- The modal container element has `aria-modal` set to `true`.

- The modal has either:
 - A value set for the `aria-labelledby` property that refers to a visible modal title.
 - A label specified by `aria-label`.
- Optional `aria-describedby` property is set on the element with the `dialog` role to indicate which element or elements in the dialog contain content that describes the primary purpose or message of the dialog. Read the full guidance at [Dialog \(Modal\) pattern](#).

★ **role="dialog" alone is not enough; always pair it with `aria-modal` and a label**
`aria-modal="true"` communicates modality but does not make background controls inert. A custom dialog must separately apply real background inertness and a focus boundary, then expose a name through `aria-labelledby` or `aria-label`.

`<dialog>` element

The native `<dialog>` element is now widely supported. When opened with `showModal()`, the browser places it in the top layer, makes the rest of the document inert, moves focus inside, constrains sequential focus, supports platform dismissal, and normally restores focus to the invoker. Product code still supplies the accessible name, initial-focus policy, close controls, and application-specific stacking behavior.

i Treat `<dialog>` as a production alternative

In this interview, implement the portaled dialog, focus management, background inertness, and ARIA behavior explicitly to demonstrate the underlying design. If a production component uses `<dialog>`, wrap its native lifecycle behind the same component API instead of independently toggling CSS visibility.

Exercise 1 of 4 Beta

A modal opens above a page. Which focus behaviors preserve a coherent interaction boundary? Select all that apply.

1 Focus the dialog container by default even when it has a title and a safe first field. ☐

2 Return focus to the trigger or another logical target after close. ☐

3

Keep keyboard focus within the open dialog and make background content inert.

☐

4

Move focus to an appropriate element after the dialog opens.

☐

5

Use dialog semantics and connect an accessible title and optional description.

☐

6

Keep DOM focus on the page and intercept Tab globally to simulate containment inside the modal.

☐

Reveal answer

Check answer ↩

Animations and transitions

If animation of the modal elements is desired and the transitions of the overlay and the contents need to be independent (e.g., the overlay fades in while the contents translate up vertically), the DOM structure will have to be changed and the contents cannot be rendered within the overlay's DOM. A structure similar to this will be required:

```
1 <div>
2   <!-- The overlay, rendered as a fixed sibling to the contents. -->
3   <div class="modal-overlay" aria-hidden="true"></div>
4   <!-- Full screen container to center the panel. -->
5   <div class="modal-contents-container">
6     <div class="modal-contents">...</div>
7   </div>
8 </div>
```

Here's an example of the entrance animation in React:

Exit transitions are non-trivial to implement properly in React because conditional rendering causes the DOM elements to be removed from the document when they are no longer needed on the page. Here's an example of a modal where the entrance and exit are both animated.

Honor `prefers-reduced-motion` by shortening or removing the transition. When duration is zero, complete the closing phase synchronously instead of waiting for a transition event that will never fire.

Exercise Beta

How should a modal close animation interact with focus and DOM removal?

- 1 Enter a closing phase, prevent new interaction, remove the dialog after the transition, and then restore focus; skip or shorten motion when requested.
- 2 Set the closing state and restore focus after a timeout equal to the current CSS duration token.
- 3 Restore focus as soon as close is requested but keep the fading dialog mounted until its transition ends.



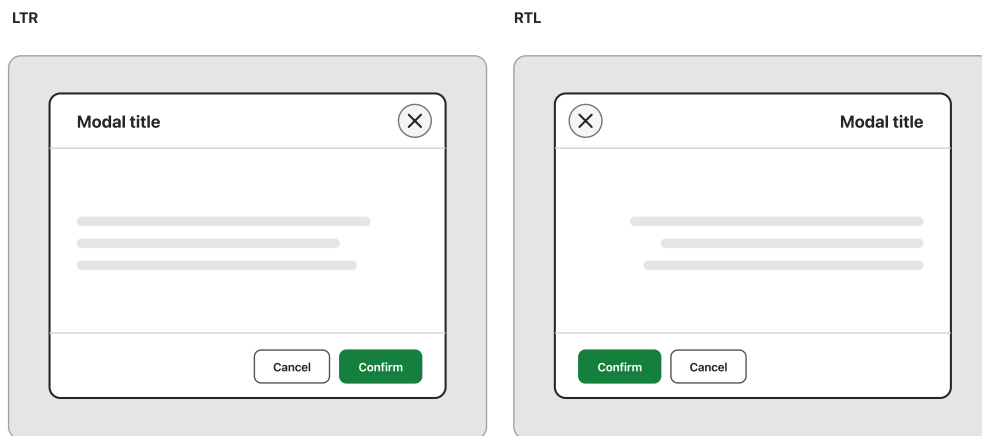
Reveal answer

Check answer ↩

Internationalization (i18n)

Since all user-facing strings are provided by the user, strings can be displayed as-is. Allow titles, content, and actions to grow or wrap for longer translations, with `Modal.Body` scrolling when the dialog reaches its viewport cap.

For RTL languages, inherit `dir` by default and use logical CSS properties. Action order follows locale and design-system policy rather than automatically flipping every element.



Exercise Beta

A modal's translated actions are longer than English and the page can run in RTL. Which changes preserve its content and interaction? Select all that apply.

- ☐ 1 Allow dialog content and buttons to wrap or scroll for longer translations.
- ☐ 2 Translate titles, action labels, validation messages, and accessible names.
- ☐ 3 Mirror the page in RTL but leave dialog content and action order forced to LTR for consistency.
- ☐ 4 Keep fixed action widths and abbreviate translations that do not fit.
- ☐ 5 Use logical spacing and alignment so layouts can mirror in RTL.



Reveal answer

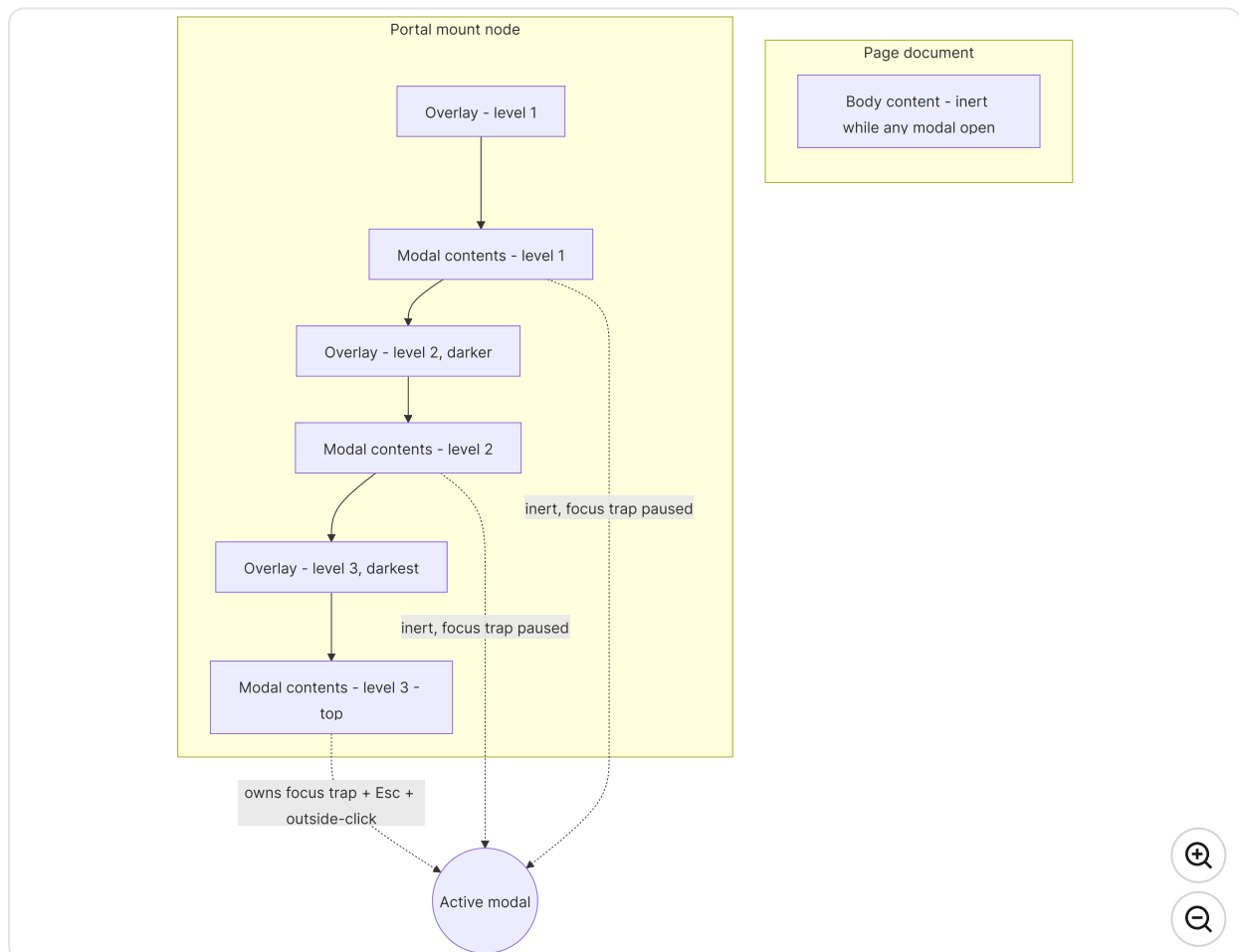
Check answer ↩

Stacked modals

It is possible for modal contents to contain triggers that present even more modals, so the following needs to be considered:

- Decide whether there should be an overlay per modal level, which will visually make the backdrop darker the higher the level of stacking.
- Dismissing a modal via the **Esc** key or clicking outside the topmost modal's contents should only dismiss the topmost modal and not all the modals.
- Dismissing a lower-layer modal should also dismiss all the modals on top of it (or make this behavior customizable).

Implementation-wise, a shared modal manager holds a stack of open modals, and only the topmost entry owns the active focus trap, the **Esc** handler, and the outside-click target. The layering below shows how overlays, contents, and focus responsibility fan out as deeper modals open.



Exercise Beta

A settings dialog opens a confirmation dialog above it. Which rules keep the stack's focus, dismissal, and document effects predictable? Select all that apply.

1

Only the topmost modal receives focus, Escape, and backdrop interactions.

☐

2

Give every dialog the same base z-index and rely on DOM insertion order to place the latest one above it.

☐

3

Keep background scroll locked until the final modal closes.

☐

4

When the top dialog closes, restore focus within the dialog beneath it.

☐

5

Broadcast Escape to every open dialog and let each decide whether to close.

☐

Reveal answer

Check answer ↩

Advanced topics

The following variations are useful follow-ups after the core modal lifecycle is correct.

- Tooltips and dropdown menus within a modal.
- Alert dialog role and ARIA pattern.

Summary

A modal is defined less by the box on the screen than by the three invisible systems wrapped around it. Portal rendering, focus management, and ARIA semantics sit behind a compound surface: `Modal`, `Modal.Header`, `Modal.Body`, and `Modal.Footer` coordinated through a root context, driven by `isOpen` and `onClose`, with `maxHeight`, `width`, `as`, and `className` for customization.

Implement the modal behavior explicitly for the interview. Render the custom dialog through a portal, make the background inert, contain focus, and route dismissal through the modal controller. Native `<dialog>.showModal()` provides many of these behaviors in production, but it is a comparison point rather than a substitute for explaining the design.

Trap focus inside the active modal. Opening the modal captures `previousFocusElement`, moves focus inside, and keeps `Tab` and `Shift + Tab` cycling within the tabbable set with edge wraparound so focus never escapes into hidden background content. `Esc` and overlay clicks both route through the same `onClose` path, and closing restores focus to the original trigger so keyboard users resume exactly where they left off. When modals stack, only the topmost owns the trap and the dismissal handlers.

Expose dialog semantics together with behavior. A custom contents container carries `role="dialog"`, `aria-modal="true"`, and an accessible name, while the manager separately applies real background inertness, pointer blocking, and focus containment. RTL layout uses logical properties, and long translations wrap or use the bounded body scroll area.

Start with the portal, focus trap, and ARIA semantics as the three concerns that make a modal reliable in production, and treat layout, animation, and theming as easier work.

Nested floating layers, exact scroll-lock restoration, transition race handling, and the full browser/assistive-technology matrix are potential follow-ups rather than requirements for the core interview answer.

References

The references below group modal and dialog sources by theme, so it is easier to map each source back to the part of the article it informed.

W3C ARIA authoring practices

This subsection anchors the dialog design in the authoritative ARIA guidance for modal and alert dialogs.

- [Dialog \(modal\) pattern | W3C ARIA](#) for the canonical roles, states, and keyboard behavior for modal dialogs.
- [Alert and message dialogs pattern | W3C ARIA](#) for the stricter alertdialog variant used for urgent messages.

Focus management and the inert attribute

This subsection collects the platform primitives that enforce focus trapping and background inertness while a dialog is open.

- [<dialog>: The Dialog element | MDN](#) for the native HTML element that handles top-layer rendering and modal semantics.
- [Inert attribute | MDN](#) for removing background content from the tab order and accessibility tree while the dialog is open.
- [A CSS approach to trap focus inside of an element](#) for a lightweight CSS-based technique for focus trapping.
- [createPortal | React](#) for rendering the dialog outside its parent DOM hierarchy so it can escape stacking contexts.

Themed implementations

These libraries ship opinionated visual defaults and are useful when comparing component API ergonomics.

- [Modal | Bootstrap v5.3](#) for a widely used CSS-first modal implementation.
- [React Modal component | Material UI](#) for a themed React modal with full keyboard and accessibility behavior.

Headless implementations

These libraries focus on behavior and accessibility without shipping styles, which is closer to what this article designs.

- [Dialog | Base UI](#) for a composable dialog with trigger, portal, backdrop, and popup parts.
- [Dialog | Ariakit](#) for an unstyled dialog with focus management and composable controls.

- [Modal | React Aria](#) for customizable modal and dialog components with accessibility behavior.

Case studies

These writeups describe how product teams ship modals and dialogs in real applications.

- [Building a Dialog for Reddit Web](#) for a production account of designing a reusable dialog system across a large web app.

Related system design articles

The articles below overlap with modal dialogs on portals, focus management, and dismissal behavior.

- [Dropdown menu](#) for overlapping concerns around focus management, portals, and dismissal semantics in overlay components.

Exercises

Exercise 1 of 13 Beta

Which requirements define the reusable modal component before its architecture is chosen? Select all that apply.

1 Allow consumers to adapt colors, fonts, spacing, and other presentation to their product. ☐

2 Support modal content on mobile, tablet, and desktop viewports. ☐

3 Decide whether consumers provide arbitrary content or use built-in title, body, close, and footer regions. ☐

4 Require every modal to use one fixed title, body, and footer template with no consumer-provided content. ☐



Reveal answer

Check answer ↩

Discussions

Normal T

B

I

U

<>

Aa

</>

Type comment here...

0/1000

Post

Report an issue

Mark complete

Subscribe to our newsletter

Enter your email

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

https://www.greatfrontend.com/questions/system-design/modal-dialog

30/31

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community

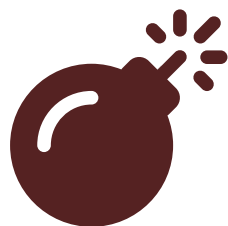


English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

• [Privacy Policy](#)

• [Terms of Service](#)



Syntax error in text
mermaid version 11.14.0